Comprehensive Chat Application HLD Guide (SDE-1 Interview Preparation)

1. Problem Statement

Design a highly scalable chat application supporting one-to-one messaging, group chat, history, presence, media sharing, notifications and fault tolerance.

2. Requirements

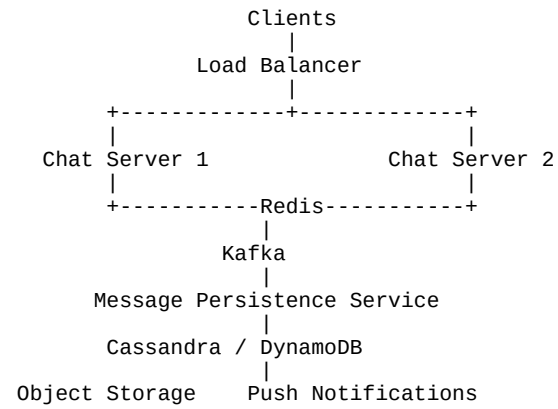
Functional: Login, messaging, history, presence, typing, read receipts, offline sync, media, groups.
Non-functional: Low latency, scalability, availability, durability, reliability, fault tolerance.

3. Capacity Estimation

Example: 100M DAU, 100 messages/day/user => 10B messages/day (~115K avg QPS, ~250K peak). Message size ~150B => 1.5TB/day (~550TB/year).

4. High Level Architecture

See diagram below.



5. Connection Flow

```
Client -> LB -> Chat Server
       -> Authenticate
       -> WebSocket Established
       -> Redis: UserID -> ServerID
```

6. Send Message Flow

```
Sender -> Chat Server
       -> Redis lookup
       -> Receiver Server
       -> Receiver
       -> Kafka
```

```
          -> DB
```

7. Offline Flow

```
       Sender -> Kafka -> DB
                   |
             Push Notification
                   |
             Receiver reconnects
                   |
            Fetch missed messages
```

8. Presence Flow

Heartbeat every 30 seconds. Redis stores ONLINE status with TTL. Expired heartbeat marks user OFFLINE.

9. Typing Indicator

Temporary WebSocket event. Never stored in database because it has no long-term business value.

10. Read Receipts

```
       Receiver opens message -> Read event -> Sender notified through WebSocket. Event may be asynchronous
```

11. Media Sharing

Upload media to object storage. Store only metadata and URL in message database.

12. Group Chat

Fan-out-on-write for small groups. Hybrid/fan-out-on-read for very large groups to reduce write amplification.

13. Database Design

Typical fields: MessageID, ConversationID, SenderID, ReceiverID/GroupID, Timestamp, SequenceNo, Status, PayloadURL.

14. API Design

```
       POST /messages/send
       GET /messages/history
       WS /connect
       POST /presence
       POST /readReceipt
```

15. Why WebSockets?

Persistent full-duplex communication eliminates repeated polling and greatly reduces latency and unnecessary requests.

16. Why Redis?

In-memory O(1) routing and presence storage. Perfect for transient state.

17. Why Kafka?

Durability, retries, asynchronous processing, traffic buffering, decoupling producers from consumers.

18. Why Cassandra/DynamoDB?

Horizontally scalable NoSQL databases optimized for massive write throughput.

19. Scaling Strategy

Stateless chat servers, Redis Cluster, Kafka partitions, DB sharding, CDN/object storage for media, multi-region replication.

20. Failure Handling

Server crash: Kafka retains messages. Redis failure: use clustered Redis. DB failure: replicas and backups. Network retry: idempotent message IDs.

21. Ordering

Partition Kafka by conversation. Maintain per-conversation sequence numbers.

22. Duplicate Messages

Every client request carries a UUID. Ignore duplicate UUIDs to make send operation idempotent.

23. Consistency

Strong durability for messages. Eventual consistency acceptable for presence, typing, last seen and read receipts.

24. Security

Authentication, TLS, end-to-end encryption (client-side), rate limiting, authorization checks.

25. Interview Answer Framework

Clarify requirements -> Estimate scale -> Draw simple design -> Introduce WebSockets -> Scale with LB

Top 40 Interview Questions with Answers

1. Q: Why not HTTP polling?
A: High latency and huge request overhead.

2. Q: Why WebSockets?
A: Persistent real-time communication.

3. Q: Why Redis over DB for presence?
A: High-frequency transient updates.

4. Q: Why Kafka?
A: Reliable asynchronous pipeline.

5. Q: How avoid message loss?
A: Durable queue + retries.

6. Q: How maintain ordering?
A: Conversation partitioning.

7. Q: Why NoSQL?
A: Write-heavy workload.

8. Q: Why not Redis for messages?
A: Not durable enough for long-term storage.

9. Q: How deliver offline messages?
A: Persist then sync on reconnect.

10. Q: How implement delete-for-everyone?
A: Soft delete + broadcast delete event.

11. Q: How implement edit message?
A: Versioning or update event.

12. Q: How support groups?
A: Fan-out strategy.

13. Q: How store media?
A: Object storage + URL.

14. Q: How scale to billions?
A: Sharding, partitioning, stateless services.

15. Q: How detect online users?
A: Heartbeat + TTL.

16. Q: How handle reconnect?
A: Resume from last acknowledged message.

17. Q: How prevent duplicates?
A: Idempotency key.

18. Q: How secure APIs?
A: Authentication + authorization + TLS.

19. Q: What if Redis fails?
A: Redis Cluster/replication.

20. Q: What if Kafka fails?
A: Retries and replicated brokers.

21. Q: Why eventual consistency?
A: Acceptable for non-critical metadata.

22. Q: What is sticky session?
A: Route same client to same server during WebSocket.

23. Q: Can load balancer route every message?
A: No; Redis mapping is needed.

24. Q: How paginate history?
A: Message ID/timestamp cursor.

25. Q: Why object storage?
A: Cheap scalable media.

26. Q: How archive old chats?
A: Cold storage.

27. Q: How search chats?
A: Separate search index.

28. Q: How block users?
A: ACL before delivery.

29. Q: How rate limit?
A: Token bucket.

30. Q: How monitor system?
A: Metrics, logs, tracing.

31. Q: How detect abuse?
A: Moderation pipeline.

32. Q: How encrypt?
A: Client-side keys.

33. Q: How backup?
A: Snapshots + replication.

34. Q: How restore?
A: Replica promotion.

35. Q: Trade-offs?
A: Latency vs consistency.

36. Q: Fan-out write vs read?
A: Write faster reads vs cheaper writes.

37. Q: Schema design?
A: Partition by conversation.

38. Q: Message lifecycle?
A: Sent->Delivered->Read.

39. Q: Why UUID?
A: Uniqueness and retries.

40. Q: Biggest bottleneck?
A: Connection management and write throughput.

Final Revision Sheet

Remember these keywords: WebSockets, Load Balancer, Stateless Chat Servers, Redis Presence, User→Server Mapping, Kafka, Cassandra/DynamoDB, Object Storage, Push Notifications, Idempotency, Conversation Ordering, Heartbeats, Fan-out, Sharding, Multi-region Replication.